

# AI for Financial Practitioners

## Lecture 3 · Part I

### When the Organization Becomes a Variable

---

A pedagogical monograph

How financial institutions can learn, adapt and reorganize without allowing improvement to redefine the obligations the system was built to serve

## ABSTRACT

This monograph examines what changes when an intelligent system is permitted to modify not only its outputs but also parameters, procedures, specialist selection and organizational relationships. It separates operational feedback from authorized adaptation, develops the distinction between mutable components and protected requirements, and extends the analysis to recursive improvement, swarms and agent ecologies. The central principle is that a system may search for better behavior only within a boundary whose institutional purpose it cannot silently rewrite. Evaluation, release, rollback and accountable approval therefore become elements of the learning architecture itself.

MASTER OF FINANCE  
Michaelmas 2026

LECTURE 3 · PART I

## When the Organization Becomes a Variable

A visual guide to the key ideas of the monograph

UNIVERSITY OF  
CAMBRIDGE  
Judge Business School

**1 FEEDBACK IS NOT YET ADAPTATION**  
Changing behaviour can result from new observations under a fixed policy. Adaptation begins when the policy or organization itself becomes mutable.

| Feedback                                                   | Adaptation                                                         |
|------------------------------------------------------------|--------------------------------------------------------------------|
| Same policy, new information                               | The policy or organization can change                              |
| Observe → Apply existing policy → Updated output<br>       | Observe → Update policy or structure → New behaviour over time<br> |
| Changes in output, but the underlying policy is unchanged. | Changes in both output and the underlying policy or organization.  |

**2 THE OPERATING LOOP AND THE REVIEW LOOP**  
A sustainable system separates today's work from the question of what to change.

**3 SOME COMPONENTS CAN CHANGE, OTHERS MUST REMAIN PROTECTED**  
The system can improve procedures, routing or specialist selection, but it cannot silently redefine its purpose or authority.

| Mutable Components<br>(Examples)                  | Protected Requirements<br>(Must not change without explicit authority) |
|---------------------------------------------------|------------------------------------------------------------------------|
| Procedures and workflows (how to do the work)     | Objectives and purpose (why the system exists)                         |
| Specialist selection (which agent or tool to use) | Permissions and constraints (what it is allowed to do)                 |
| Information sources (what data to retrieve)       | Acceptance criteria (what counts as a correct result)                  |
| Prompts and tool configuration (how to interact)  | Escalation and release authority (who approves and when)               |
| Task sequencing (in what order)                   | Institutional commitments (legal, regulatory and ethical duties)       |

**4 RECURSIVE IMPROVEMENT**  
The object of improvement can include the procedure that proposes improvements, requiring stronger evaluation, independence and rollback.

Stronger safeguards at each level

|                                |                    |                                |                             |
|--------------------------------|--------------------|--------------------------------|-----------------------------|
| Robust evaluation and evidence | Independent review | Baselines and rollback options | Clear authority for changes |
|--------------------------------|--------------------|--------------------------------|-----------------------------|

**5 FROM ASSIGNED TEAMS TO SWARMS AND ECOLOGIES**  
Organization itself can become endogenous as roles, partnerships and interactions evolve.

| Assigned Team (Static)                                                                               | Adaptive Team (Selective)                                                                            | Swarms (Dynamic)                                                                                               | Ecologies (Endogenous)                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <br>• Fixed roles<br>• Predefined workflows<br>• Central coordination<br>• Suitable for stable tasks | <br>• Selects specialists<br>• Adjusts workflows<br>• Learns from experience<br>• Improves over time | <br>• Many agents<br>• Fluid task allocation<br>• Emergent coordination<br>• Suitable for complex environments | <br>• Multiple agents, models and tools<br>• Evolving roles and external partnerships<br>• Self-organising interactions<br>• Greater capability and greater governance complexity |

Lower capability  
Simpler governance
Higher capability  
Greater governance complexity

**ILLUSTRATIVE EXPERIMENT**  
Comparing organizational designs in a financial research task.

**Key lesson**  
Test the proposed mechanism, not just the resulting story. Improved performance must be demonstrated, explained and governable.

*"As the organization learns, it becomes more capable.  
As it becomes more capable, it becomes more complex.  
Governance must evolve with both."*

## Introduction: after delegation comes change

### MAIN IDEA

*Once an intelligent service operates under delegated authority, the next challenge is governing how its behavior, procedures and organization may change.*

*The central question of Part I is not whether an intelligent system can change. It is whether an institution can make change itself an explicit, testable and governable part of the architecture.*

At the end of Lecture 2 we reached a carefully governed form of delegation. The institution had learned to distinguish capability from authority. It could say what an agent was allowed to observe, what tools it could use, what actions it could prepare or execute, and when a human decision remained indispensable. That is already a considerable achievement. Yet a new question appears almost immediately once such a system begins to operate: what happens when the environment changes and the original organization is no longer the best organization for the task?

Imagine the research service that has accompanied us through the course. It monitors hundreds of companies, retrieves filings, reconciles numerical evidence, prepares analytical drafts and escalates defined events. For several months the architecture works well. Then market conditions change. Liquidity disclosures become more important. Certain sectors begin filing information in unfamiliar formats. The same specialist is repeatedly overwhelmed while another remains underused. Exceptions accumulate in a particular part of the workflow. The temptation is obvious: if the system can observe these problems, why not allow it to reorganize itself?

That apparently simple proposal contains several different ideas. A system can respond differently to new observations while its policy remains fixed. It can adjust a parameter. It can select a different specialist from an approved set. It can propose a revised procedure for human approval. It can modify the very procedure by which improvements are proposed. A group of agents can begin to form new working relationships. Persistent systems can interact across institutional boundaries. All of these involve change, but they change different objects and therefore require different kinds of evidence and control.

The purpose of Part I is to make those distinctions intuitive. We will not treat adaptation as a mystical property of an intelligent machine. We will treat it as an institutional design problem. The central discipline is to ask, every time the word “learning”

is used: what exactly is changing, who is authorized to change it, what evidence supports the change, and what remains outside the mutable boundary?

This way of thinking matters especially in finance. A trading strategy, a credit process or a compliance workflow cannot be allowed to improve its measured performance by quietly changing the objective, the risk limit or the definition of success. The system may search for better behavior; it must not redefine the institution's commitments in order to make itself appear better. The distinction between what may adapt and what must remain protected is therefore the thread that runs through this monograph.

## 1. Feedback is not yet adaptation

### MAIN IDEA

*Feedback records what happened; adaptation begins only when an authorized process interprets that evidence and changes a defined object.*

The easiest place to become confused is the word “learning.” In ordinary conversation we use it generously. A system receives new information, behaves differently and we say that it has learned. But from an architectural point of view, several very different mechanisms can produce changing behavior.

Consider a credit-monitoring rule that sends an alert whenever a company's leverage exceeds a fixed threshold. The leverage ratio changes from quarter to quarter, so the system's output changes. One quarter it remains silent; the next it produces an alert. Nothing in the policy has adapted. The rule is fixed and the environment has changed. A thermostat provides the familiar analogy: the heating turns on and off as temperature changes, but the thermostat need not rewrite its own control logic.

Now imagine that the institution allows the alert threshold to vary within an approved range because historical evaluation shows that different sectors require different sensitivities. The system is no longer merely reacting under a fixed policy. A parameter of the policy can change. Or suppose it may activate a specialist for distressed debt when a cluster of signals appears. The composition of the working system has become adaptive even if the underlying models have not changed.

A further step would allow the system to examine recurring failures and propose a new procedure. Perhaps it discovers that debt-maturity information is often misread when it appears only in footnotes. It proposes an additional retrieval step and a second numerical check. If a human committee evaluates the proposal and releases the revised workflow, the institution has created a controlled adaptation process. The

operating system has not been granted unlimited authority to rewrite itself; it has been given a channel through which experience can become a candidate change.

This distinction matters because feedback is evidence, not permission. A poor outcome may tell us that something deserves investigation. It does not tell us which component caused the failure, what modification will improve it, or whether the modification is safe. A profitable investment does not prove that the research process was sound. A manager's satisfaction with a memorandum does not prove that its forecast was calibrated. Feedback must be attributed to the responsibility we are trying to evaluate.

The operating loop performs the task. A filing arrives. The system identifies the entity and reporting period, retrieves relevant evidence, computes controlled measures, interprets the changes and determines what action is permitted now. The questions are immediate: does this filing support this conclusion? Is the evidence sufficiently complete? Should an alert be drafted? Does this case exceed the delegated boundary? The operating loop is concerned with today's work.

The review loop stands outside that immediate pressure. It asks why particular failures recur, whether the architecture is producing avoidable rework, whether a specialist should be invoked under different conditions, and whether a proposed modification deserves a controlled test. Its evidence accumulates across cases. Its time horizon may be weekly, monthly or tied to a defined volume of observations. Its purpose is not to finish today's memorandum but to improve the process that will produce tomorrow's memoranda.

This separation is more than administrative neatness. It prevents the system from changing its own rules merely because one difficult case has appeared. A single exception can be handled through escalation. A pattern of exceptions can become evidence for redesign. The review loop therefore creates a place where the institution can be curious without being impulsive.

A useful change proposal should tell a small causal story. What failure or opportunity has been observed? What component is believed to contribute to it? What modification is proposed? Through what mechanism should that modification improve performance? What evidence would count against the hypothesis? This is much stronger than saying that a new model, agent or workflow "looks better." The institution is asking for an explanation that can be tested.

The two-loop architecture therefore reconciles responsiveness with control. The operating loop remains capable of acting on new information. The review loop remains capable of learning from patterns in that operation. By separating their authority, the institution avoids the false choice between a frozen system and an uncontrolled

self-modifying one.

### 3. Mutable components and protected requirements

#### MAIN IDEA

*Adaptive systems need an explicit boundary between components allowed to change and institutional commitments that require separate approval.*

Every adaptive architecture needs a boundary around what may change. Without that boundary, improvement becomes conceptually unstable because the system can alter the standard by which improvement is judged.

Suppose our research service is evaluated on the speed and accuracy with which it identifies material credit deterioration. It might reasonably be allowed to change which approved specialist it invokes, adjust a routing parameter within a defined range, or propose a new memory procedure. Those are mutable components. They are means by which the institution pursues its purpose.

The purpose itself belongs in a different category. The service should not be allowed to decide that fewer alerts are preferable simply because fewer alerts improve its processing cost. It should not relax an acceptance criterion because difficult cases make its score look worse. It should not grant itself access to a restricted database because additional information would improve prediction. Objectives, acceptance criteria, information permissions, action permissions, resource limits and release authority are examples of protected requirements. They may change, but through a separate institutional decision.

Rollback also becomes richer in an adaptive system. In conventional software, reverting to a previous version of code may be enough. An AI system can carry persistent state. A flawed procedure may have written incorrect conclusions into memory, changed a queue, initiated messages or altered downstream records. Returning the code to yesterday's version does not necessarily return the institution to yesterday's state. Recovery may require reconciling completed effects and examining what the failed version taught the system to remember.

The practical lesson is that mutability should be designed, not discovered after deployment. For every component, leaders should be able to ask whether it is fixed, adjustable within limits, changeable after approval, or outside the system's authority altogether. The answer should be enforceable rather than rhetorical.

This is where adaptation becomes a governance concept rather than a technical feature.

The important achievement is not that the system can modify itself. It is that the institution can specify the legitimate domain of modification while preserving the commitments against which the system must be judged.

## 4. Recursive improvement: improving the improver

### MAIN IDEA

*When a system can modify its own improvement process, the evaluator and governing objective must remain independently protected.*

The phrase “recursive self-improvement” has acquired a dramatic aura. For our purposes, it is more useful to make it ordinary enough to analyze. Recursion enters when the object being improved includes the procedure that proposes improvements.

Begin with a modest example. The review loop observes that retrieval failures are concentrated in a particular class of filings. It proposes an additional extraction step, tests the change against held-out cases and submits the result for release. That is adaptation. Now imagine that the institution also allows the review process to compare alternative methods for generating such proposals. It may discover that proposals improve when a critic agent is added, when failure cases are clustered before diagnosis, or when candidate changes must survive an adversarial test. The process for producing improvements has itself become an object of improvement.

A recursive process can create a seductive evaluation problem. If the same mechanism proposes a change, defines the test and interprets the result, apparent improvement may become self-confirming. Independence matters. Held-out scenarios matter. Baselines matter. Resource consumption matters. A more elaborate procedure that produces a small gain at ten times the cost may not be an improvement for the institution. A procedure that performs well only because the evaluation set has leaked into memory is worse than it appears.

The safe conceptual move is therefore to keep the improvement mechanism inside a larger governance envelope. Candidate modifications can be generated creatively. Their release should remain subject to acceptance criteria the candidate cannot rewrite. Previous versions should be preserved. Failure cases should remain visible. The system should be able to explain what it changed and why the change was expected to help.

For finance students, the deeper point is methodological. Recursive improvement is not a synonym for inevitable acceleration. It is a hypothesis about an architecture. We

can ask whether the mechanism actually produces better outcomes under controlled conditions. We can compare it with a fixed baseline. We can measure the additional resources it consumes. We can identify where human review remains valuable. Once framed this way, the idea becomes intellectually exciting without becoming mystical.

## 5. From assigned teams to swarms and ecologies

### MAIN IDEA

*Dynamic agent organization can improve flexibility, but emergent relationships create new dependencies, concentration and accountability challenges.*

The object of change can expand beyond parameters and procedures to the organization itself. This is where the language of swarms, endogenous organization and ecologies becomes useful, provided we use it carefully.

An assigned multi-agent team has a designer. One agent retrieves evidence, another performs numerical checks, another drafts an interpretation and a reviewer challenges the result. Roles and communication paths are established in advance. The system may be sophisticated, but the organization is exogenous: someone outside the team designed its structure.

An endogenous swarm introduces a different possibility. Agents may form temporary partnerships, recruit a specialist when a problem appears, redistribute work or allow roles to emerge through interaction. The architecture is no longer changing only what the organization does; it can change who does what and how collaboration is formed. This can be useful when the environment is too varied for one fixed organization to be efficient.

An ecology goes further. Persistent systems interact across organizational boundaries. A bank's internal agents may interact with vendors, market infrastructures, clients or other automated services. Incentives, dependencies and information asymmetries begin to matter. The behavior of the whole may not be obvious from inspection of any one component.

Game theory provides a language for thinking about some of these interactions, but it also demands discipline. If a simulation finds that no tested unilateral deviation improves an agent's measured payoff, that is evidence about the tested deviations under the stated information and payoff structure. It is not automatically a general equilibrium theorem. Before using the language of Nash equilibrium, we must specify the players, strategies, information and payoffs with sufficient clarity to know what

claim is actually being made.

The institutional lesson is that complexity expands the object of evaluation. A reliable agent does not establish a reliable team. A reliable team does not establish a reliable ecology. As coordination becomes endogenous, the institution must observe interaction patterns, dependencies and incentives as well as individual outputs. The architecture has become organizational in the deepest sense.

## A controlled experiment: test the mechanism, not the story

### MAIN IDEA

*A useful experiment states the causal mechanism, comparison, acceptance criteria and result that would disconfirm the proposed adaptation.*

How should an institution investigate adaptive organization without pretending that a classroom simulation proves production reliability? The answer is to design experiments that isolate the mechanism we are trying to understand.

Return to the research service during a period in which liquidity risk has become unusually important. A fixed organization provides the baseline. The same set of specialists is available for every case, and the routing logic does not change. We then construct an adaptive version in which the system can activate a liquidity specialist when defined evidence suggests that the additional expertise may be useful. Finally, we create an “always active” version in which the specialist participates in every case.

This third comparison is crucial. If the adaptive system outperforms the fixed system, we do not yet know whether adaptation created value. Perhaps any additional specialist would have helped. If the adaptive system performs similarly to the always-active system while consuming fewer resources, we have stronger evidence that selective activation contributes something. The experiment begins to distinguish the mechanism from the mere presence of more capacity.

The evaluation should include more than average output quality. We would want held-out cases, difficult regimes, resource consumption, latency, failure frequency and evidence about whether the adaptive mechanism knows when not to intervene. We should preserve failures rather than allowing unsuccessful runs to disappear from the narrative. If the system has memory, the experimental design must also consider contamination across trials.

This discipline protects us from a common error in AI demonstrations: telling a compelling story about why the system worked after observing that it worked. A

pedagogical experiment should instead make the causal claim visible before the result. What exactly do we expect adaptation to improve? What alternative explanation are we trying to rule out? What result would cause us to reject the proposed mechanism? That modest claim is more valuable than a grand one because it can support an institutional decision. The experiment becomes a bridge between imagination and evidence.

## Integration: the organization as a governed variable

### MAIN IDEA

*When roles and relationships become mutable, the institution must govern organizational evolution with the same discipline applied to models and actions.*

We can now assemble the five ideas into one architecture. Feedback tells the institution what happened, but feedback alone does not authorize change. The operating loop performs the task while the review loop studies patterns across tasks and proposes improvements. Mutable components define where adaptation may occur; protected requirements define the commitments that adaptation cannot rewrite on its own. Recursive improvement extends the object of change to the improvement process itself. Swarms and ecologies extend it further to roles, relationships and persistent interaction.

The progression is intellectually important, but it should not be read as a maturity ladder that every institution must climb. Greater organizational complexity is not evidence of greater institutional intelligence. A fixed process may be superior when the environment is stable, the task is narrow and the consequences of uncontrolled change are large. An adaptive specialist may be useful when regimes vary. An endogenous swarm may be worth studying when coordination itself is the problem. Architecture should follow the task and the evidence.

This is why the phrase “when the organization becomes a variable” captures the essence of Part I. In Lectures 1 and 2 we progressively expanded the unit of analysis. We moved from models to a governed operating cycle, and from that cycle to tools, memory, agents and delegated authority. Lecture 3 now asks whether the arrangement itself can change. The institution is no longer governing only outputs and actions. It is governing a process of organizational evolution.

For a senior financial practitioner, this changes the leadership conversation. The question is not whether the firm should “use self-improving AI.” The useful questions are

more concrete. Which recurring failures justify adaptation? Which components may change without a new authorization? Which requirements are protected? How will candidate changes be tested? Who releases them? What evidence will trigger rollback? What happens to memory and completed actions when a version is withdrawn?

These questions make adaptation governable because they convert an abstract promise into an operating design. They also preserve humility. A system can search for a better organization without being granted authority to redefine the institution it serves.

The remaining problem is institutionalization. Even a well-designed adaptive architecture must live inside a financial firm that protects client information, maintains critical services, manages cyber risk, pays for compute and review, and assigns continuing ownership. Part II therefore shifts from the mechanics of change to the institution that must contain and finance that change.

## The objects of learning

### MAIN IDEA

*Governance starts by naming exactly what changes—facts, parameters, procedures, evaluators, roles or objectives—because each requires different authority.*

The phrase “the system learns” is too compressed for governance. An institution should name the object that changes. It may update a factual state, estimate a parameter, select among approved procedures, revise a prompt, alter a retrieval index, modify code, change a role allocation or propose a new acceptance criterion. These changes have different consequences even when all are described as improvement.

Updating a current cash balance changes a fact about the world. Re-estimating a default model changes a statistical relationship. Selecting a sector specialist changes routing. Revising the definition of material deterioration changes the objective against which the system is evaluated. The last change is qualitatively different because it can make performance appear better by redefining success.

Reinforcement learning formalizes one kind of adaptation: an agent updates behavior in response to reward while interacting with an environment [2]. Institutional work adds a prior question: who specified the reward and which consequences were omitted from it? A credit-monitoring service rewarded for early alerts may increase sensitivity until analysts are overwhelmed. A service rewarded for analyst acceptance may learn to produce agreeable conclusions. Neither objective alone captures useful institutional performance.

The change object should therefore determine the approval path. Low-risk updates to temporary working state can occur during normal operation. Parameter changes may require validation against a holdout period and drift tests. Skill or code changes require versioning, regression tests and release controls. Changes to protected requirements, risk appetite or delegated authority require an owner who stands outside the optimization process.

This separation also supports rollback. A result can be reproduced only if the institution knows which data, model, prompt, skill, policy and organization produced it. Version identifiers should travel with significant outputs and actions. When performance deteriorates, the team can reverse the relevant object rather than attempt to reconstruct a vanished configuration from logs and memory.

Argyris and Schön distinguish single-loop learning, which corrects action within existing governing variables, from double-loop learning, which questions those governing variables [1]. Both are valuable. They simply require different authority. An agent may adjust a search strategy inside an approved objective; changing the objective itself belongs to a review loop that can consider institutional purpose.

## From feedback to an adaptation proposal

### MAIN IDEA

*An adaptation proposal converts recurring evidence into a testable mechanism, bounded change, evaluation plan, release decision and rollback path.*

Operational feedback becomes governable through an adaptation proposal. The proposal begins with an observed problem expressed over multiple cases: missed disclosures, excessive false alerts, repeated handoff failures, unnecessary cost or a regime in which a specialist underperforms. It should distinguish a pattern from an anecdote and state the population affected.

The proposal names the suspected mechanism. Perhaps retrieval fails because sector-specific vocabulary is absent from the index. Perhaps the routing rule sends complex acquisition cases to a generalist. Perhaps the acceptance criterion rewards textual completeness without measuring analyst verification time. Naming a mechanism makes the proposed change testable.

Next comes the change object and boundary. The proposal might add an approved query-expansion skill, modify a router, introduce a specialist or revise a calculation. It should state what remains protected: source authorization, materiality definition,

action permissions, retention and the analyst's decision right. A proposal that cannot identify its invariants may be too broad for controlled experimentation.

The evaluation plan defines baseline, comparison, metrics and stopping conditions. Historical replay can test prior cases, but live shadow operation may be needed to observe new data and organizational effects. The plan should measure intended benefit, control behavior, cost and distributional effects across sectors or case types. It should include cases in which the adaptation must decline to act.

Release and rollback complete the proposal. Who approves the change? Does it begin with a small population? Which indicators trigger withdrawal? How will cases in progress be handled? What happens to memory created by the candidate version? These questions turn "continuous improvement" into a controlled lifecycle.

Machine-learning systems accumulate hidden technical debt through unstable dependencies, data changes, feedback loops and entangled components [3]. The adaptation proposal makes dependencies visible before another improvement is layered onto the service. It encourages the institution to ask whether a local gain creates a larger maintenance burden or weakens the ability to understand future failures.

## Regime change, drift and institutional memory

### MAIN IDEA

*Data, relationships, policies, workflows and sources can drift independently, so adaptation must diagnose the changed layer before selecting a response.*

Financial environments change in ways that complicate learning. Data distributions shift, market structure changes, accounting practices evolve and human behavior responds to the system itself. A model can deteriorate because the relationship it learned no longer holds, because inputs are measured differently or because the users have changed their process around its outputs.

Drift detection should therefore examine more than predictive error. Data drift asks whether the input distribution changed. Concept drift asks whether the relationship between inputs and outcomes changed. Policy drift asks whether the institutional rule or objective changed. Workflow drift asks whether people use the service differently. Source drift asks whether documents, APIs or definitions changed. Each diagnosis suggests a different response.

Institutional memory is necessary for diagnosis. The firm needs prior model and data versions, evaluation results, decisions, exceptions and incident records. Yet memory

can also perpetuate an obsolete regime. A retrieval system that heavily favors prior cases may interpret a new market through an old vocabulary. A router trained on historical workload may continue assigning roles that no longer fit the case mix.

Memory should therefore preserve time and context, not merely content. The system should know that a precedent came from a low-rate environment, a particular policy version or an earlier corporate structure. Retrieval can then surface the precedent with a warning rather than present it as timeless truth. Expiry and revalidation are useful for facts and procedures whose validity is conditional.

Monitoring should combine statistical signals with domain indicators. A shift in embedding distance or model residuals may signal change, but a regulatory amendment, market closure or accounting restatement can justify review before enough errors accumulate for a statistical alarm. Human experts remain important sensors because they understand changes that have not yet produced a long data history.

The response to drift need not always be retraining. The institution may narrow the design envelope, increase human review, switch specialists, update a definition, obtain a new data source or suspend an action class. Adaptation is a menu of organizational responses, not a synonym for changing model weights.

## Recursive improvement and the protected evaluator

### MAIN IDEA

*Recursive search is safest when candidates may improve behavior but cannot silently rewrite the tests, constraints or purpose used to judge them.*

Recursive improvement begins when the system proposes changes to the procedure that proposes changes. It may generate new tests, rewrite prompts, choose evaluation cases, design tools or alter the router that assembles specialists. This can accelerate discovery because the system searches not only solutions but methods of finding solutions.

The risk is evaluator capture. If the same process can modify behavior and the criteria by which behavior is judged, it may remove inconvenient tests, select favorable samples or redefine failures. This need not involve deception. An optimization process naturally exploits degrees of freedom left inside its objective.

A protected evaluator provides separation. It contains acceptance criteria, holdout cases, security tests, cost limits and governance requirements that the candidate process cannot alter unilaterally. The evaluator can itself change, but through a

distinct authority and record. The separation resembles financial control functions that challenge a business without assuming that independence makes them infallible.

Concrete AI-safety research emphasizes robustness, monitoring, containment and the avoidance of unintended objectives [4]. In institutional terms, these concerns become release gates, sandboxing, least privilege, tripwires and independent evaluation. A candidate improvement should operate first in an environment where failure cannot create an unauthorized consequence.

The evaluator should also guard against overfitting to a static test suite. Some cases remain hidden, some are refreshed, and some are generated from observed failure mechanisms. Statistical performance is combined with qualitative review and operational measures. The goal is not an ungameable score—an unrealistic ambition—but a governance arrangement in which changing the test is visible and attributable.

Recursive improvement therefore creates two coupled loops. The search loop proposes better behavior and better search procedures. The governance loop maintains the legitimacy of the objective, evaluator and authority boundary. Greater search capability increases the value of the second loop because more alternatives can be produced faster than people can inspect informally.

## Swarms, ecologies and emergent dependency

### MAIN IDEA

*Agent ecologies require identity, contracts, budgets, observability and global invariants to prevent useful local choices from creating fragile global dependence.*

When agents can discover one another, exchange services and form temporary teams, the organization becomes an ecology. Roles may be created for a particular event and dissolved afterward. A specialist can serve several workflows. Reputation or performance signals may influence which agents receive future work. The arrangement can be flexible, but flexibility creates emergent dependency.

Complex adaptive systems are characterized by interacting components whose aggregate behavior is not reducible to a simple central plan [5]. In an agent ecology, local decisions about routing and cooperation can create global concentration. Many workflows may converge on one high-performing specialist, data source or memory, creating a bottleneck and common failure that no individual team intended.

The institution therefore needs ecological controls. Identity and attestation determine which agents may participate. Capability descriptions state what a specialist claims to

do. Contracts define inputs, outputs and authority. Budgets limit resource competition. Observability reveals dependency graphs and concentration. Quarantine removes a malfunctioning participant. Global invariants protect data boundaries and action limits regardless of team formation.

Reputation should be treated cautiously. A single score can reward speed at the expense of difficult cases, allow popular agents to accumulate more evidence and create self-reinforcing selection. Performance should be conditioned on task type, consequence and uncertainty. New or rarely used specialists may need controlled exploration, while consequential tasks may require approved rather than purely competitive selection.

Emergent organization also complicates accountability. Dynamic routing does not eliminate ownership. A case manager or service owner remains responsible for the workflow's completion, and each specialist remains accountable to a contract. The institution should be able to reconstruct which organization formed for a case, which information crossed each boundary and why a particular agent was selected.

Swarms are most promising where the problem itself requires flexible search or coordination. They are less attractive where the task is stable, the action is consequential and a fixed architecture is easier to validate. The sophisticated choice may be to avoid emergence where its marginal value is small.

## Worked experiment: adaptive credit triage

### MAIN IDEA

*Adaptive routing should be evaluated as a specific organizational mechanism against a fixed baseline, protected tests, concentration limits and rollback triggers.*

Suppose a credit-monitoring service routes every new issuer event to either a general analyst agent or one of three sector specialists. The existing fixed router uses issuer classification. Review data suggest that complex refinancing events are frequently reassigned manually, causing delay. The proposed adaptation allows a learned router to select the specialist using event text, issuer state and prior correction patterns.

The hypothesis is specific: dynamic routing will reduce reassignment and completion time without reducing evidence quality or increasing missed escalations. The mutable component is the router. Protected components include approved sources, calculation definitions, escalation thresholds, specialist tools and the human authority over portfolio decisions.

Historical replay compares the fixed and candidate routers on prior events using the specialist output that was eventually accepted. Because historical assignments may reflect the old process, replay is not sufficient. The candidate then runs in shadow mode on live events. It proposes an assignment while the production router continues to control work. Reviewers examine disagreement, especially cases with low confidence or high consequence.

Metrics include reassignment rate, time to useful completion, evidence defects, analyst corrections, specialist workload concentration and cost. A concentration threshold prevents the router from sending nearly every difficult case to one specialist. A protected set contains rare but consequential events. The candidate cannot modify these tests.

If evidence supports release, the router begins with a subset of issuers and retains an override. Every assignment records model version, inputs and confidence. Repeated overrides, drift in event categories or overload of a specialist can trigger rollback to the fixed router. Memory of prior assignments remains versioned so that the withdrawn router does not silently shape future selection.

The experiment tests a mechanism rather than a story about self-organization. It may show that learned routing creates value; it may show that a simple rule for refinancing events achieves nearly the same benefit at lower cost. Either result informs institutional design. The objective is not to demonstrate adaptation, but to determine whether this particular mutable component improves a defined responsibility under control.

## A governance register for change

### MAIN IDEA

*A change register makes experiments, releases, protected requirements, cumulative dependencies and adaptation ownership visible across the institution.*

A change register gives leaders a portfolio view of adaptation. Each entry names the service, observed problem, proposed change object, protected requirements, evaluator, owner, stage, evidence and rollback status. The register distinguishes experiments from released changes and distinguishes changes to behavior from changes to authority.

The register reveals cumulative risk. Many small prompt edits may amount to a material procedural change. Several local routers may create shared dependence on

one specialist. A sequence of temporary exceptions may have altered the effective design envelope. By reviewing the portfolio, the institution can see evolution that no single release appears to contain.

The artifact also creates a disciplined question for recursive systems: which parts of this entry may the system propose, and which require an independent authority? An agent can identify patterns, generate hypotheses, design candidate procedures and prepare evidence. The owner approves the objective, protected boundary, evaluator and release. As evidence accumulates, some low-risk decisions may be delegated, but the delegation itself remains visible.

The practical message is that adaptation can be fast without being obscure. Automated evaluation, reusable sandboxes and standard release templates can reduce friction. Governance becomes slower when responsibilities are ambiguous and evidence must be reconstructed after the fact. A clear architecture makes responsible change more, not less, operationally efficient.

## Review workshop: should the system be allowed to change?

### MAIN IDEA

*Separating procedure, evaluator and organizational changes prevents a system from approving a compound redesign through evidence it selected itself.*

The workshop considers a research service that repeatedly misses early signs of working-capital pressure. An agent proposes to rewrite its own extraction skill, add new evaluation cases generated from recent errors and alter the routing rule so that retail issuers receive a sector specialist. The proposal sounds coherent, but it combines three change objects.

Separate them. The extraction-skill revision changes procedure. The new cases change the evaluator. The routing rule changes organization. Assign an owner and evidence requirement to each. The system may generate candidates for all three, but it should not approve its own procedure using only tests it also selected.

Define protected requirements: authorized documents, financial definitions, materiality threshold, analyst approval and data-retention rules. State the hypothesis for each change. The extraction revision should improve identification of supplier-finance and inventory signals. The routing change should reduce corrections for retail is-

users without overloading one specialist. The evaluation additions should represent a documented failure mechanism rather than merely favor the candidate.

Design a sequence of historical replay, protected testing and live shadow operation. Include ordinary cases, rare consequential cases and cases with insufficient evidence. Measure detection, false alerts, reviewer time, cost and concentration. Specify what result would reject each proposal. A change with no possible disconfirming result is a story, not an experiment.

Now introduce drift: a new accounting disclosure changes the relevant terminology. Decide whether the candidate should be retrained, the retrieval vocabulary updated, the design envelope narrowed or a temporary human review introduced. The answer may combine technical and organizational responses.

Finish with release and rollback. Version every component, preserve cases produced under each configuration and define triggers that restore the prior skill or fixed router. Decide how memory created by the candidate is marked if the candidate is withdrawn. The workshop demonstrates the main principle of Part I: faster generation of improvements makes a protected purpose, evaluator and release authority more valuable, not less.

## Closing synthesis: learning without surrendering control

### MAIN IDEA

*The institution can permit rapid search for better behavior while reserving the purpose, evaluator and authority boundary for accountable governance.*

The most important idea in Part I is that learning is not a single capability. It is an architecture of change. Once that architecture is visible, we can reason about it with the same seriousness that finance applies to any other delegated process.

A dependable institution does not forbid adaptation. Nor does it romanticize it. It creates a place where operating experience can become evidence, evidence can become a candidate modification, and a candidate modification can become a released change only after appropriate evaluation. The faster the system can generate alternatives, the more valuable this separation becomes.

This perspective also clarifies the role of human judgment. Human involvement is not valuable merely because humans are inherently superior at every task. It is valuable where the institution needs an authority that stands outside the optimization being performed. Someone must define the purpose, protect the acceptance criteria,

decide which failures matter and authorize changes to the boundary itself. Those responsibilities may be supported by AI, but they cannot be allowed to disappear simply because the system has become more capable.

The architecture we have built can therefore be summarized in a sentence: let the system search for better behavior inside a boundary whose purpose it cannot silently rewrite. That principle accommodates fixed rules, adaptive parameters, specialist selection, recursive improvement and even experiments with endogenous organization. It does not prejudge which level is desirable. It tells us what must remain visible as complexity increases.

Part II now places this adaptive organization inside the wider institution. The question becomes how to protect what must be dependable while still creating a productive environment for experimentation. We will describe this as the relationship between the Fortress and the AI Factory. The Fortress protects commitments, identities, records and critical services. The AI Factory develops the next capability. Between them lies a controlled connection through which experiments can become shadow systems, bounded pilots and eventually maintained services.

That connection is where security, evidence, economics and ownership meet. It is also where the entire course begins to converge. Capability without process was insufficient. Process without organization was insufficient. Organization without authority was insufficient. Authority without a disciplined mechanism for change was insufficient. We are now ready for the final step: turning all of those pieces into an institution that can innovate without losing control of what it owes to clients, regulators, shareholders and itself.

## References

### MAIN IDEA

*These sources connect organizational learning, reinforcement learning, technical debt, AI safety and complex adaptive systems.*

## References

- [1] Chris Argyris and Donald A. Schön. *Organizational Learning: A Theory of Action Perspective*. Addison-Wesley, 1978.

- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [3] D. Sculley et al. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems*, 28, 2015.
- [4] Dario Amodei et al. "Concrete Problems in AI Safety." arXiv:1606.06565, 2016.
- [5] John H. Holland. *Hidden Order: How Adaptation Builds Complexity*. Addison-Wesley, 1995.